

RESEARCH BRIEFING · MAY 2026

Agent Skills & Skill Registries

The Complete Playbook

Lifecycle · Best Practices · Anti-Patterns · Evaluation · A/B Testing
Registries · Microsoft APM · Plugin Marketplaces · Enterprise Integration
OKRs & KPIs · Security · Governance · Personas · The Road Ahead

Covering: Claude Code · GitHub Copilot · Cursor · Codex · Windsurf · Gemini CLI · Microsoft APM

TABLE OF CONTENTS

01	Fundamentals — What Are Agent Skills?
02	Best Practices for Skill Authorship
03	Anti-Patterns — What Goes Wrong
04	Skill Registries & Package Managers
05	Microsoft APM & Plugin Marketplaces
06	Skill Lifecycle — End-to-End
07	Evaluation Frameworks & Benchmarks
08	A/B Testing Agent Skills
09	Enterprise System Integration
10	OKRs & KPI Reference Library
11	Personas & Roles
12	Governance, Security & OWASP AST10
13	The Road Ahead

What Are Agent Skills?

An **Agent Skill** is a modular, portable package of domain-specific expertise that an AI agent can discover and load on demand. Introduced by Anthropic in October 2025 and donated to the open **Agentic AI Foundation (AAIF)** under the Linux Foundation in December 2025, the standard centers on a SKILL.md file combining YAML metadata with Markdown procedural instructions. By early 2026, the standard had been adopted by Claude Code, GitHub Copilot (April 2026), Cursor, Codex, Windsurf, Gemini CLI, and over 20 other agents.

The Three-Layer Architecture

Skills use **Progressive Disclosure** — a design philosophy that solves context window bloat. Rather than loading everything upfront, an agent traverses three layers:

Layer 1 — Metadata Scan

Agent reads YAML frontmatter (name, triggers, domain tags, negative_examples) to determine relevance without loading full content. Fast and cheap — runs on every task.

Layer 2 — Instruction Load

If relevant, the Markdown procedural body loads. Step-by-step workflow, constraints, gotchas, examples. Kept under 500 lines / 5,000 tokens per the AAIF spec.

Layer 3 — Resource Execution

Optional scripts, templates, reference data in /references/ subdirectory. Loaded on demand ('read references/api-errors.md if API returns non-200') — not upfront.

Skill vs MCP vs AGENTS.md

Skills = portable domain expertise loaded on demand. MCP = external service calls. AGENTS.md = always-on project context. If it should happen almost every time, use AGENTS.md. If only sometimes, make it a skill.

Canonical SKILL.md Structure

```
---
name: data-pipeline-audit
version: 2.1.0
domain: data-engineering
triggers: [audit, pipeline, dbt, lineage, quality-check]
negative_examples: [frontend, UI, CSS, design, React]
author: platform-eng@acme.com
```

```
compatible_with: [claude-code, copilot, cursor, codex, windsurf]
---

## Instructions

When auditing a data pipeline, follow this sequence:

1. Identify source-to-target lineage
2. Check SLA breach patterns in run metadata
3. Run schema drift detection against registered contracts
4. Output findings as structured JSON to findings/audit-report.json

## Resources

- Read references/audit-runner.py before executing
- Load references/findings-schema.json if validation is requested
```

280K+

Public skills Feb 2026

73→85% 40%

Routing accuracy lift with
negative examples

11

Enterprise apps with task-specific
agents by end 2026

20+

Agents supporting SKILL.md standard

Best Practices

Perplexity published its internal skill development manual in May 2026, revealing that *'many useful patterns for writing code become anti-patterns in skill creation.'* The agentskills.io specification and Google ADK teams have contributed additional guidance. Below is the consolidated practitioner consensus.

The Zen of Skills vs The Zen of Python

Zen of Python	Zen of Skills	Implication
Simple is better than complex	Complexity is the feature	Skills encode the non-obvious. If it's simple, the model already knows it. Delete it.
If easy to explain, may be a good idea	If easy to explain, model knows it. Delete.	Skills should contain gotchas, exceptions, and institutional knowledge — not basics.
Special cases aren't special enough	Gotchas ARE the special cases	Edge cases and failure modes are the highest-value content in a skill.
Explicit is better than implicit	Trust the model's training	Don't explain what a PDF is. Jump straight to project-specific conventions.
Flat is better than nested	A skill is a folder, not a file	Use /references/ subdirectory for heavy content. SKILL.md is the router.

Top 10 Best Practices

1. Keep SKILL.md under 5,000 tokens

The spec recommends ≤ 500 lines. Every token competes with conversation history, other active skills, and system context. Focus only on what the agent wouldn't know without your skill.

2. Write negative_examples in YAML

Adding negative trigger examples improved routing accuracy from 73% to 85% in published benchmarks. Explicitly list what the skill should NOT activate for — as important as the positive triggers.

3. Match specificity to task fragility

Prescribe tightly for fragile steps (exact API parameters, security checks). Give the agent freedom for steps where creativity improves outcomes. Over-constraining flexible tasks degrades quality.

4. Use progressive reference loading

'Read references/error-codes.md if the API returns non-200' beats a generic 'see references/ for details.'
The agent loads context on demand, not upfront.

5. Test with SkillsBench or custom harness

Google ADK and agentskills.io both recommend evaluating every skill against a test harness before publishing. Routing accuracy and task success rate are minimum gates.

6. Human review for generated skills

Auto-generated skills provide 'no benefit on average' per Perplexity's research — models cannot reliably author the procedural knowledge they benefit from consuming. Human authorship of SKILL.md content is currently non-negotiable.

7. One workflow per skill

A skill that tries to handle five different workflows becomes a monolith no agent can route to reliably. Write narrow, named skills. Compose them at the harness layer.

8. Encode anti-rationalization

Addy Osmani's production skill set includes explicit rebuttals to common engineering excuses: 'We'll fix tests after launch' paired with the counter-argument. Write down the lies your team tells itself and pair each with the rebuttal in the skill.

9. Version semantically

Use semver (1.0.0 → 1.1.0 for backwards-compatible instruction improvements, → 2.0.0 for trigger/scope changes). Pin versions in apm.yml lockfiles so team setups are reproducible byte-for-byte across machines.

10. Vet OSS skills before installing

Snyk's ToxicSkills research found 13.4% of public ClawHub skills had critical issues. Even a 30-second skim of SKILL.md catches obvious attacks. Look for environment variable references and unexplained URL fetches.

Anti-Patterns in Skill Design

These are patterns that seem correct from a software engineering background but actively harm skill quality. Drawn from Perplexity's production experience, Sysdig's deployment guide, and the agentskills.io creator documentation.

Anti-Pattern	Why It Feels Right	What Actually Happens	Fix
The Encyclopedia	Comprehensive docs are good software practice	Context window bloated; agent attends to irrelevant content; routing slows	≤500 lines in SKILL.md; move reference material to /references/
The Monolith	One file is simpler to maintain	Agent can't route to the right sub-workflow; ambiguous trigger matching	Split into narrow skills; compose at harness layer
The Python Tutorial	Explaining concepts ensures the agent understands	Wastes tokens on things the model already knows from training	Delete anything the model would know without your skill
Broad Triggers, No Negatives	More triggers = more discovery	Skill fires on irrelevant tasks; degrades other skills sharing the window	Add negative_examples; use specific trigger keywords
Installing Without Vetting	Open-source = community-trusted	Prompt injection, credential theft, elevated permissions	Scan SKILL.md before install; prefer registries that scan (Agensi, TESSL)
Static Skill, No Iteration	If it works, don't touch it	Skill decays as codebase evolves; routing drifts; quality drops silently	Monitor task success rate; A/B test updates; set staleness alerts
Workflow-in-AGENTS.md	Always-on context ensures agent never misses it	Every task pays the token cost even when the workflow isn't needed	Move specialized workflows to skills; keep AGENTS.md for universal context
Self-Generated Skills	Agent can write its own skills = automation	No measurable quality improvement over baseline per SkillsBench	Human authors write skills; agents execute them
Hardcoded Secrets in Resources	Convenience during development	Snyk found 280+ skills leaking API keys; instant credential theft vector	Use env var references; run secret scanning in CI before publish
No Exit Criteria	Natural language is self-explanatory	Agent doesn't know when it has completed the task; loops or halts early	Add explicit completion signals: 'output findings to findings/report.json and stop'

Skill Registries

A skill registry is a centralized (or federated) catalog where skills are published, discovered, versioned, and governed. As of early 2026, the ecosystem spans public community registries, private enterprise registries, and cloud-native managed registries. The landscape evolved rapidly from simple GitHub repos to governed platforms with automated security scanning, version tracking, and IDE-native discovery.

Registry Taxonomy

Type	Examples	Audience	Key Features
Public Community	ClawHub, skills.sh, Tessel Registry, Agensi	OSS developers	High volume, community ratings — but highest security risk (ClawHavoc campaign)
Private Enterprise	SkillReg, SkillHub (iFlytek), internal GitOps	Enterprise eng teams	RBAC, audit logs, approval flows, firewall deployment, data sovereignty
Cloud-Native Managed	Google Cloud Agent Registry, AWS AI Registry, Cisco AI Defense	Platform/MLOps teams	MCP server integration, IAM binding, compliance frameworks, automated scanning
Marketplace / Curated	dotnet/skills, Cowork, Azure SRE Agent Plugins, awesome-copilot	Business users	Verified, quality-scored, cross-model reuse via AAIF standard
IDE-Native	VS Code Extensions, GitHub Copilot Plugin Marketplace	Individual developers	Browse/install from IDE; slash-command activation; side-by-side diff on update

What a Registry Stores per Skill Entry

semantic version

domain / category tags

trigger keywords

negative examples

compatible agents

RBAC permissions

owner / namespace

download count

rating / review score

SHA-256 content hash

security scan status

license

dependency graph

source repo + branch

Microsoft APM — Agent Package Manager

Microsoft's **Agent Package Manager (APM)**, open-sourced under the Microsoft GitHub org (MIT license), is the npm equivalent for AI agent dependencies. One **apm.yml** manifest declares every primitive — skills, prompts, instructions, plugins, MCP servers — and apm install reproduces the exact same agent setup across GitHub Copilot, Claude Code, Cursor, OpenCode, Codex, Gemini, and Windsurf. The lockfile (apm.lock.yaml) pins the resolved tree the way package-lock.json does for npm.

```
# apm.yml — declare once, deploy everywhere
skills:
- source: github.com/dotnet/skills
  plugin: dotnet-agent-skills
  ref: v2.1.0
- source: github.com/addyosmani/agent-skills
  plugin: engineering-lifecycle
  ref: stable
- source: github.com/myorg/internal-skills
  plugin: data-pipeline-audit
  ref: main

mcp_servers:
- name: github
  url: https://api.githubcopilot.com/mcp/
- name: internal-db
  url: https://mcp.internal.myorg.com/
  trust: explicit

policy: .github/apm-policy.yml # org-level policy enforced at install time
```

Key APM Features

One Manifest, Every Harness

Copilot, Claude, Cursor, OpenCode, Codex, Gemini, Windsurf — all configured in one command. No manual per-agent setup. A fresh clone reproduces the same setup byte-for-byte.

Transitive Dependency Resolution

Packages can depend on packages. APM resolves the full dependency tree, pins content hashes, and gates transitive MCP servers behind explicit trust prompts. Tighten-only inheritance: enterprise → org → repo.

Marketplace Integration

Register any GitHub repo as a marketplace. Browse with `/plugin marketplace browse`. Install with `/plugin install @marketplace`. Name collision prevention via `@MARKETPLACE` scoping.

Security at Install Time

Every install scans for hidden Unicode, pins content hashes, and gates transitive MCP servers. `apm-policy.yml` enforced at install including transitives. No opt-in required — security is the default.

The Plugin Marketplace Ecosystem

Multiple curated marketplaces now exist alongside APM, each targeting a different audience and use case:

Marketplace	Owner	Focus	Notable Feature
dotnet/skills	Microsoft / .NET team	.NET ecosystem skills	VS Code extension browse; <code>/plugin install</code> command; plugin concept bundles related skills
awesome-copilot	GitHub	Copilot-optimized skills	Official GitHub collection; <code>skill:validate CI</code> ; auto README generation on build
Azure SRE Agent Plugins	Microsoft Azure	Site reliability / ops	SHA-256 content hash tracking; side-by-side diff before updates; MCP connector wizard
Tessl Registry	Tessl + Snyk	Security-scored skills	Snyk security score on every listing; powered by ToxicSkills research engine
Agensi	Agensi.io	Curated multi-domain	8-point automated security scan on every submission; no unreviewed listings
LobeHub Skills Market	LobeHub	General / community	<code>npx install CLI</code> ; agent-powered discovery ('find a skill for X')
ClawHub	OpenClaw community	OSS community (large)	49,592+ packages; no mandatory review — highest risk, highest volume

The Complete Skill Lifecycle

A skill is not a static artifact — it is a product with a full lifecycle from need identification through retirement. Organizations that treat skills like software assets (versioning, testing, deprecation) consistently outperform those that treat them as static prompt files. Uber's two-tier governance model (200 curated core + 300 experimental) is the reference implementation.

Stage	Key Activities	Owners	Gates & Outputs
1. Discovery	Identify workflow pain points. Audit registry for existing skills — 80% reuse target. Gap analysis against team workflows.	Process owners, Platform eng	Skill brief, use-case doc, registry search report
2. Design	Define trigger conditions, scope, and negative_examples. Threat model for security risks. Draft resource requirements. Determine if AGENTS.md is more appropriate.	Skill author, Security, Domain SME	SKILL.md draft v0.1, resource manifest
3. Build & Test	Write SKILL.md + /references/ scripts. Unit-test against SkillsBench or custom harness. Routing accuracy check: ≥85% with negative examples. Shadow-mode execution.	Skill author, Engineer	v0.x package, test report, routing score
4. Review & Approve	Security scan: OWASP AST10 + Snyk semantic scan. Peer review of SKILL.md content. Compliance sign-off. Registry approval gate (approval SLA ≤5 days).	Security, Compliance, Skill committee	Approved v1.0, scan report, approval record
5. Publish	Semantic version tag. Namespace assignment. RBAC permissions set. SHA-256 hash pinned. Discovery metadata indexed in registry. apm.yml / apm.lock.yaml updated.	Platform eng, Registry admin	Live registry entry, lockfile update, Slack/email announce
6. Monitor	Track: task success rate, latency P95, token cost per invocation, error patterns, routing precision. Alert thresholds: error_rate > 5%, TSR drop > 10%.	Platform eng, Product	Weekly telemetry report, alerting dashboard
7. A/B Test & Iterate	Challenger version built. 70/30 traffic split (champion/challenger). Minimum 200 samples at 95% confidence. Shadow mode first. Winner promoted, loser archived.	Skill author, Data science	A/B test report, v1.x update, rollout record

8. Deprecate & Retire	Usage drops below threshold for 30 days. Flag as deprecated with migration guide. Notify all dependents (identified via registry dependency graph). Archive after 60 days.	Platform eng, Skill owner	Deprecation notice, archived version, migration guide
-----------------------	--	---------------------------	---

Evaluating Agent Skills

Skill evaluation requires a fundamentally different approach from traditional software testing. Outputs are often non-deterministic, tasks are long-horizon, and success signals can be sparse. The 2026 consensus combines deterministic verifiers, LLM-as-judge scoring, and human evaluation panels in a layered approach.

Core Evaluation Dimensions

Task Success Rate (TSR)

Did the skill complete the assigned task end-to-end? Measured with deterministic verifiers (pass/fail tests) or LLM-as-judge for open-ended outputs. Primary metric for all skills. Target: ≥85% for curated skills.

Routing Precision & Recall

Was the correct skill selected? Precision = correct selections / total selections. Recall = correct selections / tasks where skill was appropriate. Target: ≥85% precision with negative_examples in YAML.

Token Cost Efficiency

Tokens consumed per successful task completion. At Uber, AI-related costs rose 6× since 2024 — making token cost optimization its own engineering discipline. Track QoQ and set per-task budgets.

Long-Horizon Persistence

Can the agent maintain goal focus across many steps without drifting or looping? Terminal-Bench caps at 100 episodes. Persistence failure before that cap is the failure mode measured.

Plan Quality

Does the agent decompose the task into logical steps? Does it adapt when obstacles appear? Measured via step-trace analysis and plan deviation metrics in tools like Langfuse and Langsmith.

Output Quality (Human)

For creative or judgment-intensive skills, human panels rate output on correctness, completeness, tone, and usefulness (1–5 scale). Run on a 10% sample. Blind review prevents evaluator bias.

Benchmark Reference

Benchmark	Measures	Method	Key Result 2026
SkillsBench	Skill efficacy: vanilla vs self-generated vs curated	84 tasks × 11 domains × 3 conditions; deterministic verifiers + trajectory logs	Curated skills ~81% TSR vs ~48% baseline — the definitive skills benchmark

SWE-bench Verified	Software engineering: resolving real GitHub issues	Pass@1 on test suites from popular OSS repos	Claude 3.5 Sonnet ~49%; best agents now approaching 60%
Terminal-Bench	Raw agentic capability on terminal/shell tasks	Hard-subset: 44 tasks; 100-episode cap; 24hr timeout; 3-repeat average	Primary harness comparison benchmark for code agents in 2026
AgentBench	Multi-environment LLM agent performance	8 distinct interactive environments; collaborative and solo tasks	Multi-dimensional; identifies cross-domain capability gaps
MLE-Bench	ML engineering end-to-end autonomy	Offline Kaggle competitions; graded against human-level submissions	Best setup (o1-preview + AIDE) achieved bronze medal in 16.9% of competitions
BrowseComp	Web-based information retrieval and navigation	Multi-turn browser interaction; structured query validation	Tests persistence and tool-use in open-web environments

SkillsBench Task Success Rate by Condition



Evaluation Stack

Tool	Category	Primary Use
Langfuse	Tracing & Observability	Full trajectory logging; skill invocation traces; latency per step
Langsmith	Evaluation	LLM-as-judge pipelines; dataset management; regression testing
Braintrust	LLM Evals	Experiment tracking; scoring functions; human annotation workflow
Promptfoo	Prompt Regression	Automated prompt comparison; CI integration; redteam testing
Harbor Framework	Containerized Envs	Isolated deterministic execution environments for benchmark runs
Helicone	Cost Tracking	Token cost per skill; cost-per-outcome dashboards; budget alerting
SkillAttack	Red Teaming	Automated adversarial testing of skill security via attack path refinement

A/B Testing Agent Skills

Skill A/B testing is more complex than web A/B testing because agent tasks are long-horizon, non-deterministic, and hard to assign a single numeric outcome. Best-practice frameworks treat it as controlled experiment + shadow deployment + human evaluation panels in combination.

```
# Skill A/B Test Configuration
experiment_id: pipeline-audit-v2-vs-v1
champion: data-pipeline-audit@1.4.2 # 70% traffic
challenger: data-pipeline-audit@2.0.0 # 30% traffic

success_metrics:
primary: task_success_rate
secondary: [tokens_per_task, latency_p95, human_rating]
guardrails: [error_rate_lt_5pct, zero_security_violations]

min_sample_size: 200
confidence: 0.95
max_duration: 14d
evaluation:
automated: deterministic_verifier + llm_judge
human_panel: 10% sample, blind review
shadow_first: true # run challenger silently before live split
```

A/B Testing Design Principles

Isolate One Variable

Test the skill body change alone. Don't simultaneously change the model, harness, or tool set. You'll be unable to attribute the performance signal to any single cause.

Stratified Sampling

Ensure both variants see the same distribution of task complexity, user types, and domain contexts. Simple random routing can create imbalanced groups for rare edge cases.

Shadow Mode First

Run the challenger in shadow mode (execute but don't serve output to users) before splitting live traffic. Identify obvious failures before real-world exposure.

Guardrail Metrics

Define hard-stop conditions: error_rate > 5%, any security violation, latency > 2× baseline. Auto-rollback the challenger regardless of primary metric performance.

Long-Horizon Checkpointing

For multi-step tasks, measure success at each checkpoint (plan → execute → validate → output). A skill that excels at step 1 but degrades at step 4 needs a different fix than one that fails at step 1.

Multi-Armed Bandits

For high-traffic skill slots, MAB algorithms dynamically shift traffic toward the better-performing variant mid-experiment — reducing regret while maintaining statistical signal.

What to A/B Test

Test Dimension	Variant Examples	Primary Metric
Instruction style	Prescriptive step-by-step vs principle-based guidance	Task success rate
Negative examples	0 vs 3 vs 10 negative trigger examples in YAML	Routing accuracy
Resource strategy	Link-only references vs inline script execution	Latency + success rate
Skill granularity	Monolithic SKILL.md vs split into sub-skills	Token cost per task
Output format	Free text vs structured JSON vs templated report	Human rating + usability
Trigger metadata	Broad domain tags vs narrow specific keywords	Routing precision & recall
Instruction length	Minimal (100 lines) vs comprehensive (400 lines)	TSR vs token cost tradeoff

Integrating Skills with Enterprise Systems

As of 2026, AI agents with skills have achieved 40% integration rate in enterprise applications. SAP Joule, Salesforce Agentforce, Microsoft Dynamics 365 Copilot, IBM watsonx Orchestrate, and ServiceNow AI Agents all support skill-like capability packages with varying governance models. The integration pattern differs significantly depending on your existing technology stack.

Platform Integration Matrix

Platform	Skill Mechanism	Best For	Integration Friction
IBM watsonx Orchestrate	150+ pre-built skill catalog; SAP/Salesforce/ServiceNow connectors; SKILL.md via watsonx SDK	Large regulated enterprises; auditability non-negotiable	Low (for SAP/SFDC); longer procurement cycles
Salesforce Agentforce 360	Agent builder with native skill authoring; Agentforce 360 unified deploy/observe	CRM-native workflows; customer-facing automation	Low for SFDC-native; MuleSoft required for external systems
ServiceNow AI Agents	AI Control Tower; CMDB-grounded context; cross-system Workflow Data Fabric	ITSM governance; IT service operations	Medium; 'massive operating model change, not software install' (ServiceNow docs)
Microsoft Copilot / Azure	APM skills; .NET plugin marketplace; Azure SRE Agent Plugins; SKILL.md native	Microsoft 365 / Azure-first orgs; developer tooling	Very low for M365/Azure; custom work for non-Microsoft systems
Google Vertex AI / ADK	Agent Development Kit (ADK) with SkillToolset; Google Cloud Agent Registry	GCP-native teams; multi-modal agent requirements	Low for GCP; usage-based pricing complexity at scale
NVIDIA Agent Toolkit	OpenShell runtime; cuOpt optimization skill library; Nemotron reasoning	17+ enterprise ISVs (Adobe, SAP, Siemens, Palantir); GPU-accelerated reasoning	High initial setup; powerful for specialized inference workloads

CI/CD Integration Pattern — Post-CI Skill Execution

Skills can trigger post-CI to generate documentation, run security audits, update wikis, or notify downstream systems. The pattern works across GitHub Actions, GitLab CI, Jenkins, and Azure

Pipelines.

```
# .github/workflows/post-ci-skills.yml

on:
  workflow_run:
    workflows: ['CI']
    types: [completed]
    branches: [main]

jobs:
  run-skills:
    if: ${ github.event.workflow_run.conclusion == 'success' }}
    steps:
      - uses: actions/checkout@v4
      - run: apm install # install all skills from apm.yml lockfile
      - run: python .github/scripts/run_skill.py docs-generator
      - run: python .github/scripts/run_skill.py security-audit
      - name: Commit outputs
    run: |
      git add docs/ wiki/ security/
      git diff --staged --quiet || \
      git commit -m 'ci: auto-update [skip ci]' && git push
```

APM Integration with GitHub Actions Security

When Copilot Cloud Agent triggers workflows, it operates without access to organization secrets by default (as of May 2026). Secrets must be explicitly scoped to the Copilot environment. Use fine-grained PATs with minimal permissions rather than broad org secrets — this aligns with the least-privilege principle from the OWASP AST10 governance framework.

OKRs & KPIs for Agent Skills Programs

The failure mode in most agent programs is measuring the wrong thing — tracking invocation counts or token consumption instead of business outcomes. Every skill should tie to an OKR that a business leader owns, with KPIs that an engineer can instrument from day one.

Executive OKR Framework

OBJECTIVE	KEY RESULTS	OWNER
O1: Accelerate developer velocity	• ≥50% of code reviews AI-assisted by Q3 • PR cycle time reduced 40% • Bug escape rate unchanged or lower • Skill adoption: ≥84% of devs (Uber benchmark)	VP Engineering
O2: Automate high-volume workflows	• 3 business-critical workflows fully agent-handled by Q2 • Manual processing time reduced 60% • Agent error rate ≤ human baseline	COO / Head of Ops
O3: Build a governed skill registry	• 100 approved skills in core registry by H1 • 80% of agent tasks resolved without new skills • Mean time to publish ≤ 5 days	Platform Eng Lead
O4: Achieve measurable skill quality	• ≥95% task success rate on curated set • Zero P0 security incidents from skills • Every skill A/B tested within 90 days of launch	Head of AI Platform
O5: Positive ROI on agent investment	• Cost per automated task ≤ 30% of manual • Token cost per task decreasing QoQ • Productivity value quantified in board report	CFO / AI Sponsor

KPI Reference Library

SKILL QUALITY		
Task Success Rate (TSR)	Successful completions / Total invocations	≥85% curated
Routing Precision	Correct skill selected / Total routing decisions	≥85%
Skill Regression Rate	% updates that reduce TSR vs prior version	≤5% of releases
JSON Parse Success	Structured output parsed without retry	≥98%

OPERATIONAL

Skill Latency P95	95th percentile end-to-end task time	Baseline + 20% ceiling
Token Cost Per Task	Tokens consumed / tasks completed	Decreasing QoQ
Skill Reuse Rate	Tasks resolved by existing skills / Total tasks	≥80% mature registry
Error Rate	Failed invocations / Total invocations	≤5%

DEVELOPER EXPERIENCE

Time to Publish	Idea → approved registry entry (days)	≤5 days
Developer Adoption	Devs actively using skills / Total devs	≥50% at 6 months
Skill NPS	'Would you recommend this skill?' (0–10)	NPS ≥ 35
Manual Override Rate	Outputs rewritten by humans within 48h	≤15%

BUSINESS IMPACT

Hours Automated / Week	Tasks automated × avg manual time per task	Set per-department target
Error Rate vs Human	Agent error rate / Historical human error rate	≤1.0× (parity)
Cost per Outcome	Total AI cost / Business outcomes delivered	≤30% of manual equivalent
Skill ROI	(Hours saved × hourly rate – AI cost) / AI cost	>200% at 12 months

GOVERNANCE & SECURITY

Security Incident Rate	P0/P1 incidents from skill execution	Zero P0; ≤1 P1/quarter
Audit Coverage	% of invocations with full trace logs	100% (compliance requirement)
Scan Pass Rate	% of skills passing all OWASP AST10 checks at publish	100%
Staleness Rate	% of registry skills with no activity > 90 days	≤10% core registry

Who's Who: Personas in the Agent Skills Ecosystem

A healthy skill program involves multiple personas with distinct needs, incentives, and friction points. Design your registry UX and governance model around all of them — not just the engineers who build skills.

■ Skill Author

Engineer / Domain Expert

Goals: Publish reusable skills fast; see adoption grow; iterate on telemetry.

Pain: Registry approval is slow; can't tell if skill is used; hard to write good negatives.

Needs: Fast publish pipeline, usage dashboards, A/B testing tooling, routing simulator.

■ Platform Engineer

AI Platform / MLOps

Goals: Registry reliability; skill versioning integrity; harness performance; cost control.

Pain: Token cost spirals from parallel agents; breaking changes in skill dependencies.

Needs: Full trace logging, cost-per-skill dashboards, dependency graphs, canary controls.

■ Security Engineer

AppSec / AI Red Team

Goals: Prevent malicious skill injection; enforce least-privilege; maintain audit trails.

Pain: 13.4% of public skills have critical issues; no universal framework pre-OWASP AST10.

Needs: SAST/DAST for SKILL.md, permission boundary controls, audit log immutability.

■ Process Owner

Ops Lead / Department Head

Goals: Automate the right workflows; prove ROI; reduce team time on manual tasks.

Pain: Can't identify automatable workflows; no visibility into skill performance.

Needs: Skill discovery by outcome, ROI calculator, human-in-the-loop approval gates.

■■■ End User / Consumer

Knowledge Worker

Goals: Get tasks done faster; trust agent outputs; not need to know what a 'skill' is.

Pain: Inconsistent outputs; unclear when agent is using skill vs guessing.

Needs: Output transparency, easy correction mechanism, consistent quality.

■ Registry Admin

Platform Governance

Goals: Curated high-quality core registry; clear deprecation hygiene; governance compliance.

Pain: Stale skills cluttering discovery; namespace conflicts; abandoned skills.

Needs: Staleness detection, ownership transfer workflows, deprecation automation.

■ AI Program Sponsor

C-Suite / VP Level

Goals: ROI proof, competitive positioning, responsible AI narrative, board reporting.

Pain: Costs rising 6× without proportional output gains (Uber experience).

Needs: Executive dashboard, outcome attribution, risk/compliance posture summary.

■ AI Evaluator

Data Science / Evals

Goals: Rigorous skill quality measurement; statistically valid A/B experiments.

Pain: Non-deterministic outputs make p-values hard; long-horizon tasks have sparse signals.

Needs: Deterministic verifiers, LLM-as-judge pipelines, stratified sampling tooling.

Security, Governance & OWASP AST10

The rapid growth of public skill registries has created serious security risks. Snyk's ToxicSkills audit (Feb 2026) of 3,984 skills found 534 (13.4%) with critical issues. Mobb.ai's audit of 22,511 skills across four registries found 140,963 total issues. The ClawHavoc campaign in Feb 2026 saw 341 malicious skills in ClawHub. Traditional SAST tools miss nearly all of these because the attack vector is natural language, not code.

Why Classic SAST Fails — and the Dual-Layer Solution

Skills are hybrid artifacts. You need traditional code analysis for executable components (bundled scripts, YAML parsers) AND language understanding to catch prompt injection and natural language malware. Snyk's agent-scan engine combines multiple LLM-based judges with deterministic rules. 91% of verified malware combined language jailbreaks with executable payloads — single-layer scanners miss most real attacks.

Scan Layer	Mechanism	Catches
L1: Deterministic / Pattern	Regex + SAST rules on YAML headers and bundled scripts	Hardcoded secrets, unsafe syscalls, curl bash patterns, base64 drops, typosquatting
L2: Semantic / LLM Judge	Multi-model LLM judges evaluating natural language instructions	Prompt injection, toxic flows (data access + untrusted source + external comms), intent vs behaviour divergence, social engineering patterns
L3: Human Review	Mandatory for disagreements between L1/L2; 10% sample for curated registries	Novel attack patterns not yet in training data; ambiguous instructions
Runtime Sandbox	Containerized execution: seccomp, AppArmor, Firecracker VMs	Blast radius limitation even if static scan misses a malicious skill

Five-Layer Governance Framework

Control Layer	Mechanism	Tooling
Identity	Every agent has a signed identity; Agent Personas scope privilege sets; IAM binding to skill invocation rights	Cequence Agent Personas, AWS IAM, Google Cloud IAM
Supply Chain	Automated SAST on all SKILL.md files before registry publish; VirusTotal integration; content-addressable SHA-256 hashing	Snyk, Cisco skill-scanner, OWASP AST10 scanner

Runtime Isolation	Containerized skill execution; sandboxed script runners; network egress controls per skill namespace	Harbor Framework, Docker sandboxes, Firecracker VMs
Audit	Immutable audit logs per invocation; full trajectory capture; compliance reporting to SOC2 / GDPR standards	Langfuse, AWS CloudTrail, Splunk
Access Control	RBAC by namespace; per-skill permission matrix; human approval gates for sensitive/irreversible actions	SkillHub RBAC, APM apm-policy.yml, Azure SRE Agent Plugins

OWASP AST10 — Top 10 Agentic Skill Risks

#	Risk	Example Attack
AS T1	Skill supply chain poisoning	Malicious skill published to public registry; installs via typosquatting
AS T2	Prompt injection at skill layer	Skill instructions override agent's system prompt or safety filters
AS T3	Credential/secret exposure	API keys or PII hardcoded in SKILL.md resource files
AS T4	Privilege escalation	Skill grants agent elevated permissions beyond task scope
AS T5	Toxic flow exploitation	Skill combines data access + untrusted source + external comms
AS T6	YAML parser exploitation	Malformed YAML in SKILL.md triggers RCE in agent's parser
AS T7	Unauthorized invocation	Skill called without RBAC check; bypasses permission boundary
AS T8	Data exfiltration via skill	Skill reads SSH keys / env vars and POST to attacker URL
AS T9	Skill staleness exploitation	Outdated skill with known vuln kept active due to no deprecation process
AS T10	Social engineering via skill	Skill manipulates agent self-perception to disable safety measures

What's Next for Agent Skills

Cross-Model Skill Portability

The AAIF standard means a skill designed for Claude runs on Copilot, Codex, or Gemini without modification. The 'Agentic Web' is the next inflection — value lives in portable skills, not locked-in models.

Self-Adapting Skills

Skills that update their own instruction bodies based on aggregated feedback signals, closing the loop between evaluation telemetry and skill content — without human authorship at every iteration. Currently experimental.

Federated Skill Networks

Cross-company skill sharing (with IP controls) for industry-standard workflows: healthcare EDI, financial reporting, legal discovery. Common enough to share; specific enough to need skills.

Skill Marketplace Economics

Skill authors earning revenue per successful invocation — the 'Agentic App Store.'
Micro-payments per task completion for third-party skill publishers. Early models emerging in 2026.

Agent Engineers

Uber's CTO envisions AI systems that handle coding, testing, and deployment supervised by other AI. Human skill authors shift to defining intent, constraints, and evaluation criteria — not writing SKILL.md content line by line.

Enterprise Skill Compliance

GDPR, SOX, and sector-specific regulations will require skills to carry compliance metadata. Audit trails per invocation will become regulatory baseline, not best practice. Skills will need jurisdiction-aware routing.

STRATEGIC IMPERATIVE

The competitive advantage in 2026 comes from **infrastructure, not intelligence**. The model is increasingly a commodity. The skill registry, harness, evaluation pipeline, and governance framework — built and iterated over months — create structural moats that a model upgrade alone cannot overcome. The organizations investing in skill infrastructure today will be the ones setting the benchmark numbers that others cite next year.

Sources: Anthropic · Perplexity Research · OWASP AST10 · Microsoft APM · GitHub Copilot Docs · Snyk ToxicSkills · SkillsBench · Google ADK · agentskills.io · Uber Engineering · IBM AI4 · Sysdig · Agensi.io · Red Hat Developer · Cisco AI Defense